

一、选择题（10 道题，每道 1 分）

1. 计算机系统层次结构中，（ ）是软硬件系统的交界面，系统程序员与硬件设计者都能看到。

- A. 操作系统
B. IS A 指令集体系结构
C. 系统函数库
D. 微程序
E.

【参考答案】A

【学长点评】从应用问题到机器语言程序的每次转换所涉及的概念都属于软件范畴，机器语言程序所运行的计算机硬件和软件之间需要一个桥梁，这个软件和硬件之间的界面就是指令集体系结构，但可以把操作系统看成是应用程序和硬件之间插入的一层软件。

2. 执行程序的组成, 对于 C 标准函数库中的函数, 不能采用 () 连接方式。

- A. 操作系统加载 B. 静态库连接
C. 运行过程中连接 D. 编译时 E.

【参考答案】A

【学长点评】对于 C 标准函数库的函数，不能采用在操作系统链接的方式进行。

3. Linux 操作系统的内核采用 () 编码。

- A. ASCII B. UTF-8 C. UTF-16 D. UTF-32 E.

【参考答案】B

【学长点评】Linux 操作系统内核采用 UTF-8 编码。

4.C 语言中 int 与 float 所表示的数据个数, 哪个一个多 ()

- A. int 多 B. float 多 C. 与编译时采用 m32/m64 模式有关
D. 一样多 E.

【参考答案】A

【学长点评】由于 float 中包含 NaN 等类型数据，而 int 型数据中无重复数据，共可表示 2^{32} 个数据，所以表示的个数比 int 少。

5. Linux 下 C 程序中出现整数除法错，如除以 0 等，会输出（ ）并退出程序。

- A.Data overflow B.Floating point exception
C.NULL D.空白行 E.

【参考答案】B

【学长点评】当试图除以 0 的时候，或者当一个除法指令的结果对目标操作数来说太大了时候，就会发生除法错误，Unix 不会试图从除法错误中恢复，而是选择终止程序，Linux 通常会把除法错误报告为浮点异常。

6. 软件工具 objdump 反编译的汇编语言中某个操作数 1024(RIP) 最可能是 ()。

- A. 参数 B. 局部变量 C. 静态局部变量 D. 全局变量

【参考答案】C

【学长点评】1024（RIP）所指数据应为 GOT 全局偏移表中的数据，GOT 表会为每个全局变量建立条目。

7. 进程的状态不包括 ()。

- A. 运行 B. 睡眠 C. 停止 D. 终止

【参考答案】B

【学长点评】从程序员的角度，进程总是处于运行、停止和终止这三个状态中。

8. 计算机系统中下列存储器最快的是（ ）。

- A. L1 cache B. TLB C. 动态内存 DRAM D. SSD 固态硬盘

【参考答案】A

【学长点评】寄存器速度最快，随后是 L1 级缓存，之后是主存，辅存速度最慢，TLB 中存放的为 PTE。

9. 父进程与其创建的子进程的堆栈（ ）。

- A. 物理存储空间上完全独立 B. 虚拟空间内存共享
C. 虚拟内存空间独立 D. 物理储存空间上完全共享

【参考答案】D

【学长点评】fork 函数执行后，两个进程有相同的用户栈、相同的本地变量值、相同的堆、相同的全局变量值、以及相同的代码，都会映射到同一个物理空间，如果对数据进行修改会发生写时复制。

10. 程序 helloworld! 执行第一条机器指令时，正确的是（ ）。

- A. 所有代码与数据都已由操作系统加载到物理内存
B. 此指令是 call main
C. 会产生缺页中断
D. 此指令已经在物理内存中

【参考答案】C

【学长点评】第一次执行 helloworld 程序指令代码未从磁盘导入主存，会发生缺页中断，需要将代码数据导入主存。

二、填空(5 道题，每道 2 分)

1. C 语言 `int x=-2;` 则 `&x` 地址处依次存放的 4 个字节为（ ）。

【参考答案】fe ff ff ff

【学长点评】int 型数据占 4 字节，2 的表示为 ff ff ff fe，由于按照小端方式存放，则低有效位存放在低地址处，故存放顺序依次为 fe ff ff ff。

2. 链接重新定位时，指令 `400800: e8 ( ) callq 400810`

【参考答案】0b 00 00 00

【学长点评】e8 说明该指令为 callq 指令，后面为需要重定位的值，取完指令后 PC 的地址为 400805，需要跳转的目的地为 400810，故偏移量为 5，可得重定位的值为 0b 00 00 00

3. TLB 俗称快表，是（ ）的缓存

【参考答案】页表

【学长点评】TLB 快表是页表的缓存。

4. Linux 下程序运行时，在键盘上输入 Ctrl-c，会产生（ ），终止程序。

【参考答案】SIGINT

【学长点评】当进程在前台运行时，如果键入了 Ctrl+C 则会发送一个 SIGINT 信号给前台进程组中的每个进程。

5. 十进制的有符号常数，由（ ）转换成二进制补码。

【参考答案】编译器

【学长点评】有符号数由编译器转换为二进制补码。

三、问答题(4 道题, 每道 5 分)

1. 请按步骤算出 float 数 0, 1 的二进制表示, 规格表示, 阶码, 位数。其在内存从低地址到高地址的 4 个字节内容(16 进制)。

【参考答案】

考点出处: IEEE754 单精度浮点数表示法

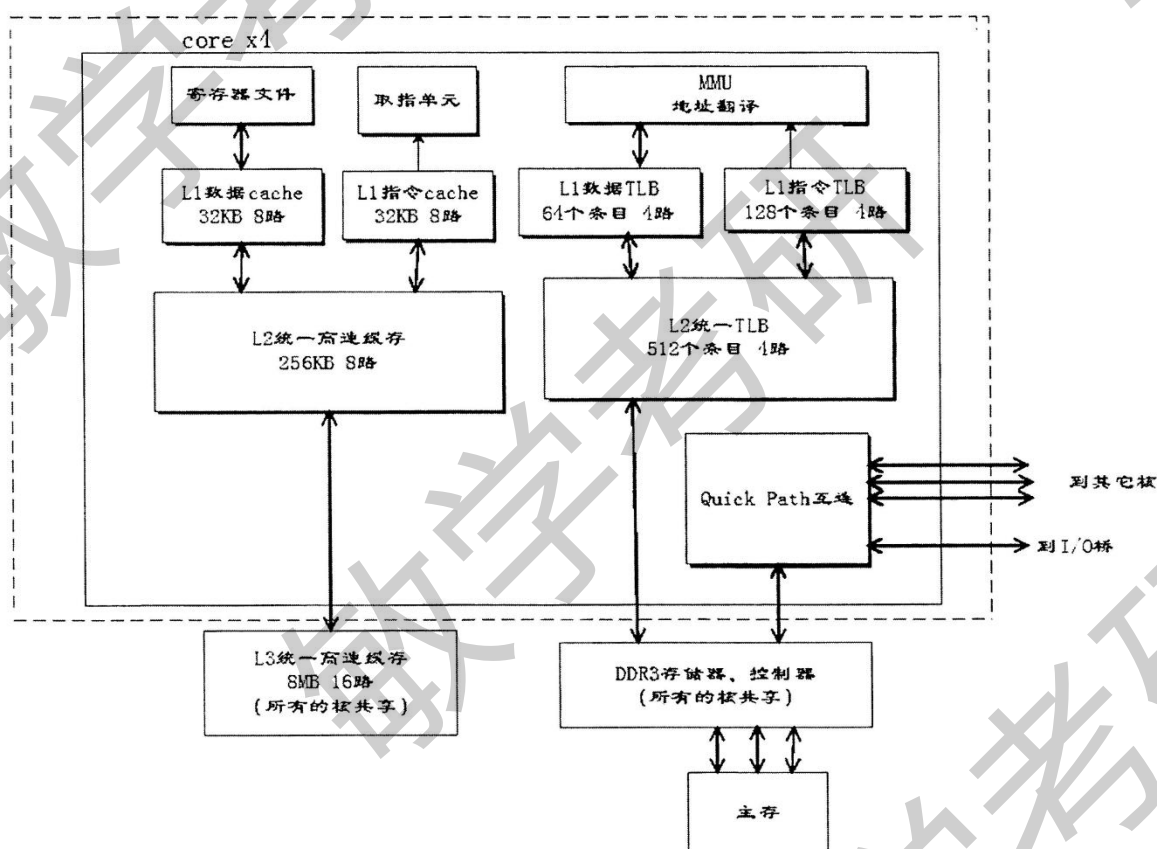
考频分析: 常考

难易程度: 一般

复习建议: 熟悉单精度浮点数的表示

【学长点评】将数据根据 IEEE754 进行表示, 1 位符号位, 8 位阶码, 23 位尾数, 正数故符号位为 0, 0.1 转为二进制为循环小数, 阶码为 0111 1011, 尾数为 10011...001100 最后结果为 3DCCCCCH。

2. Intel I7CPU 的虚拟地址 48 位, 物理地址 52 位, 每一页面 4KB。其内部结构如图所示:



分析如下项目:

- (1) 虚拟地址中的 VPN 占_____位; 其一级页表为_____项。
- (2) L1 数据 TLB 的组索引位数 TLBI 为_____位。
- (3) L1 数据 Cache 组共_____组。
- (4) 用物理地址访问 L1 数据 Cache 时, Cache 标记 CT 占_____位。

【参考答案】 36, 512, 5, 64, 40

【学长点评】 (1) 虚拟地址为 48 位, 页大小为 4KB, 页内地址为 12 位, 则 VPN 为 36 位

(2) 页大小为 4KB, 页表项为 8B, 则共有 512 项

(3) 128 个条目, 4 路, 则共有 32 组, 即 5 位

(4) 32KB 缓存, 每块大小为 64B, 8 个为一组, 共有 64 组

(5) 物理地址为 52 位，页内地址和组索引共 12 位，余下 40 位为标记。

3. 分析如下子程序，写出其对应的 C 语言源程序（函数名，参数名可自定义）

```
40114a mov    (%rsi),%rax
40114c neg    %rax
40114e mov    (%rdi),%rdx
401150 neg    %rdx
401152 mov    %rdx, (%rsi)
401154 mov    %rax, (%rdi)
401156 ret
```

【参考答案】第一个参数存放在 rdi 中，第二个参数存放在 rsi 中，并分析汇编指令。

```
int swap(int *x, int *y)
{
    int a,b;
    a=*y;
    a=-a;
    b=*x;
    b=-b;
    *y=b;
    *x=a;
    return 0;
}
```

4. 下列子程序存在着缓冲器溢出漏洞，请分析漏洞产生的原因，说明如何攻击？

```
0x401152 push %rbp
0x401153 mov %rsp, %rbp
0x401156 sub $0x28,%rsp
0x40115a mov %rdi,%rsi
0x40115d lea -0x20(%rbp),%rdi
0x401161 callq 0xC001030<strcpy@plt>
0x401168 lea -0x20(%rbp),%rdi
0x40116a callq 0x401040<puts@plt>
0x40116f leaveq
0x401170 retq
```

【参考答案】本题主要是由于 strcpy 函数存在安全隐患，需要两个参数一个为需要写入的位置，一个为读取的位置，如果写入超出界限就会引发缓冲区溢出。在 401153 处为该函数分配了一个 40 字节的栈帧，在 40115d 处将 -0x20(%rbp) 作为调用函数的参数，即以该地址作为传送字符串的首地址，但如果字符串长度超过了 0x20 便会发生缓冲区溢出，修改了函数的返回地址，使得程序发生异常。

四、设计题（3 道题，每道 10 分）

1. 在 Y86-64 的 SEQ 顺序结构 CPU 设计中，采用取址、译码、执行、访存、写回更新 PC 共六个阶段，请写出 ret 每一个阶段的微操作，在 PIPE 五段流水线 CPU 设计中，怎么避免 ret 指令的控制冒险？

【学长点评】ret 指令每个阶段的微操作如下图所示，可以通过插入气泡的方式来避免 ret 指令的

控制冒险，流水线必须暂停直到 ret 指令达到回写阶段。

阶段	通用
	ret
取指	$icode: ifun \leftarrow M_1[PC]$ $valP \leftarrow PC+1$
译码	$valA \leftarrow R[\%rsp]$ $valB \leftarrow R[\%rsp]$
执行	$valE \leftarrow valB+8$
访存	$valM \leftarrow M_8[valA]$
写回	$R[\%rsp] \leftarrow valE$
更新 PC	$PC \leftarrow valM$

2. 向量内积计算的相关程序如下。

```
/*向量的数据结构定义 */
typedef struct{
int len; //向量长度，即元素的个数
float *data; //向量元素的存储地址
} vec;

/*获取向量长度*/
int vec_length(vec *v){return v->len;}

/* 获取向量中指定下标的元素值，保存在指针参数 val 中*/
int get_vec_element(*vec v, size_t idx){
if (idx >= v->len)
return 0;
*val = v->data[idx];
return *val;
}

/*计算向量内积*/
void inner0(vec *v, vec *u, float *sum){
long int i;
*sum = 0; //初始化为 0
for (i = 0; i < vec_length(v); i++) {
float val;
*sum = *sum + get_vec_element(v,i)*get_vec_element(u, i);
}
}
```

请面向 c 编辑器，进行初步优化，以提高程序的性能，编写优化后的函数 innerr1，并说明优化依据。

【学长点评】分析代码中可以优化的地方，并进行优化，例如 for 判定中每次都计算长度很浪费

时间，可以将长度放入临时变量中。

【参考答案】将 vec 长度提前算好并放入临时变量中，可以减少函数调用次数，避免每个循环的边界检查，同时将结果累积在临时变量中，具体优化代码如下所示。

```
void inner1(vec *u, vec *v, doubt *dest)
{
    long i;
    long length = vec_length(u);
    doubt temp = IDENT;
    for(i=0;i<length;i++)
    {
        temp = temp + get_vec_element(u,i) * get_vec_element(v,i);
    }
    *dest = temp;
}
```

3. 针对上体的函数 inner0，采用 Inert i7cpu, 其中有两个带地址计算的数据加载器，2 个浮点乘法器，一个浮点加法器。U, v 的元素是 4 的倍数，采用 2*2 循环展开方式，编写面向 cpu 友好的子程序 inner2. 这个 inner2 程序是性能最优的吗？为什么？怎么获得面向本 CPU 最优的程序？

【学长点评】采用 2*2，将循环内展开可得如下程序，性能仍可提升，可以通过继续扩大展开来提升整体性能，比如变成 8*8 展开等。

【参考答案】

```
void inner2(vec *u, vec *v, doubt *dest)
{
    long i;
    long length = vec_length(u);
    long limit = vec_length(u)-1;
    doubt x0 = IDENT;
    doubt x1 = IDENT;
    for(i=0;i<limit;i+=2)
    {
        x0 = x0 + get_vec_element(u,i) * get_vec_element(v,i);
        x1 = x1 + get_vec_element(u,i+1) * get_vec_element(v,i+1);
    }
    for(;i<length;i++)
        x0 = x0 + get_vec_element(u,i) * get_vec_element(v,i)
    *dest = x0 + x1;
}
```

计算机网络

一、选择题：

1. TCP/IP 参考模型中，实现端到端数据可靠传输的层是（ ）

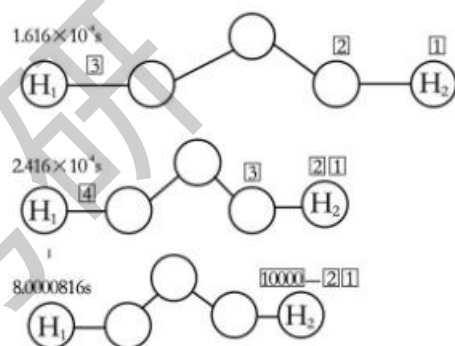
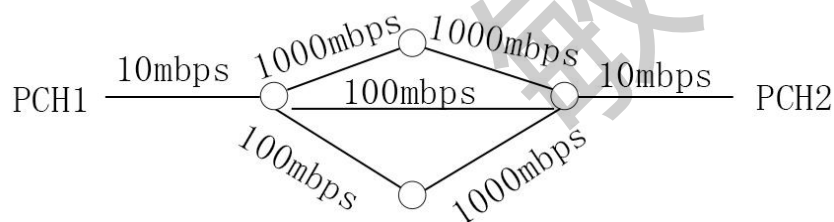
- A. 应用层 B. 传输层 C. 网络层 D. 数据链路层

【参考答案】B

【学长点评】主要考察对 TCP/IP 中各个层次的了解，比较基础，考察的就是知识点，记住就得分了

2. 下图所示网络 H1 欲向 H2 发送大小为 10MB 的文件，则从 H1 开始发送算起到 H2 收到为止，采用报文交换方式和分组交换方式（分组长度 100B）所用的时间至少分别为（ ）

- A. 16.16s, 8s B. 16.8s, 8s C. 2.1s, 1s D. 2.02s, 0.1s



【参考答案】A

【学长点评】本题考查分组交换和报文交换，首先计算报文交换需要的最少时间，选择上面的路，到第一个路由器需要 $10\text{MB}/10\text{Mbps}=8\text{s}$ ，到达上面路线的中间路由器需要 $10\text{MB}/1000\text{Mbps}=0.08\text{s}$ ，到达最后一个路由器需要 $10\text{MB}/1000\text{Mbps}=0.08\text{s}$ ，到达目的主机需要 $10\text{MB}/10\text{Mbps}=8\text{s}$ ，一共需要 $8+8+0.08+0.08=16.16\text{s}$ 。现在已经可以选出答案了，如果在考试就直接做下一道题，节约时间。接下来计算分组交换需要的最少时间，还是选择上面那一条路，经过下图的分析，就可以得出答案了。

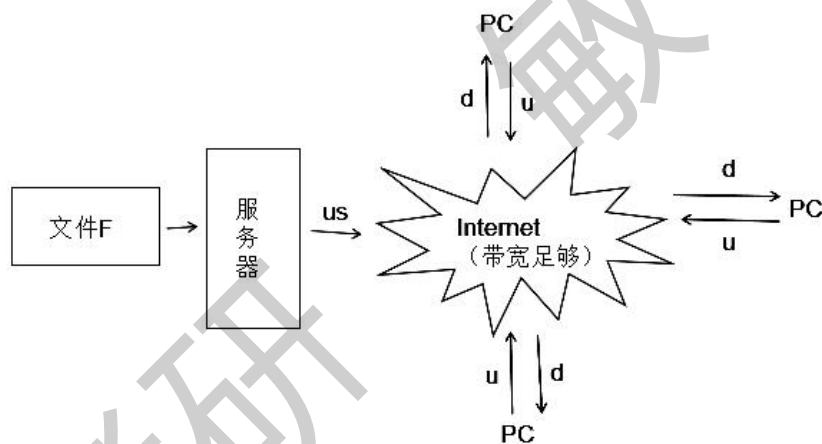
3. 如果浏览器分别采用非流水的 HTTP1.1 和支持并行 TCP 连接的 HTTP1.0, 请求一个引用 4 个 JPEG 图片的 web 页，则至少需要的时间分别是 ()

- A. 3RTT, 4RTT B. 5RTT, 3RTT C. 6RTT, 4RTT D. 6RTT, 10RTT

【参考答案】C

【学长点评】考察对 HTTP1.1 和 HTTP1.0 的了解，审题和做题都要仔细。首先判断采用非流水的 HTTP1.1 需要的时间，因为是非流水的，所以在收到前一个响应后才能发出下一个请求。建立 TCP 连接用去一个 RTT，接收网页内容用去 1 个 RTT，然后 4 个图片分别访问四次，用去 4 个 RTT，一共 6 个 RTT；再看采用支持并行 TCP 连接的 HTTP1.0 需要的时间，题中并没有说是几个并行 TCP，但是问题是求至少需要的时间，所以说不妨考虑至少有 4 个并行 TCP，那么对四个图片的访问就只需要使用 2 个 RTT，接着考虑建立 TCP 连接和获取网页内容需要的 2 个 RTT，总共需要 4 个 RTT。

4. 下图为 1000 个客户进行文件转发的应用示意图，其中 $F=1\text{MB}$, $us=1000\text{Mbps}$, $d=10\text{Mbps}$, $u=1\text{Mbps}$, 注 ($M=10^6$)，若分别采用 C/S 分发方式和 P2P 分发方式，则完成文件分发所需时间分别为多少 ()



- A. 8s, 4s B. 8s, 0.8s C. 1s, 0.5s D. 0.8s, 0.008s

【参考答案】A

【学长点评】考察 C/S 分发方式和 P2P 分发方式，做题的时候注意将 MB 里面的 Byte 换算成 bit，计算上传速率之和要记得加上服务器上传到互联网的速度。显然文件在服务器上，而且不需要考虑拥塞等状况。首先考虑采用 C/S 分发方式需要的时间，1000 个客户共需要从服务器得到的数据总量是 $1000 \times 1\text{MB}$ ，服务器不停的向各个客户传送数据，直到各个客户都收到文件 F，需要的时间是数据总量/ $u_s = 1000 \times 8 / 1000 = 8$ 秒，客户下载文件的时间为 $F/d = 1\text{MB} / 10\text{Mbps} = 0.8$ 秒，选择最大的时间作为完成文件分发所需的时间，则为 8 秒。再考虑采用 P2P 分发方式需要的时间，服务器至少将文件 F 的全部内容（每一个 bit）传送一次到互联网，因为其他客户可以代替服务器向其他客户（他们都是对等方）分发文件 F，所需时间为 $F/u_s = 1\text{MB} / 1000\text{Mbps} = 0.008$ 秒，客户下载文件的时间为 $F/d = 1\text{MB} / 10\text{Mbps} = 0.8$ 秒，文件分发的最少时间不能小于客户下载的时间，文件分发的最少时间为数据总量/上传速率之和 $= (1\text{MB} \times 1000) / (1000\text{Mbps} + 1\text{Mbps} \times 1000) = 4$ 秒，选择这是这个时间中最大的作为完成文件分发所需的时间，则为 4 秒。

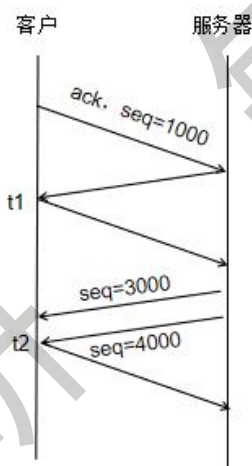
5. 主机甲通过一条链路连接主机乙，链路带宽为 10Mbps，单向传播延时为 $7\mu\text{s}$ ，若甲采用 GBN 协议向乙发送帧长为 1000B 的数据帧，乙采用帧长为 250B 的确认帧进行确认，数据帧需要为 4 比特编号，则甲的最大信道利用率为（ ）

- A. 10% B. 80% C. 85% D. 100%

【参考答案】B

解析：数据帧为 4 比特，则可发送 15 个数据帧，发送时间为 $15 \times 1000 \times 8 / 10\text{Mbps} = 12\mu\text{s}$ ，单个数据帧的时间为 $1250 \times 8 / 10\text{Mbps} + 7 \times 2 = 15\mu\text{s}$ ，故最大信道利用率为 80%。

6. 服务器通过一个 TCP 连接向客户发送数据的部分过程如图，若服务器均已最大段长 $\text{MSS} = 1000\text{B}$ 发送 TCP 段，则客户在 t_1 时刻和 t_2 时刻发送给服务器 TCP 段正确的确认序号分别是（ ）



A. 1000, 4000

B. 2000, 2000

C. 3000, 4000

D. 2000, 5000

【参考答案】D

【学长点评】考察 TCP 协议的特点。TCP 提供提供可靠交互的服务，通过 TCP 连接传送的数据无差错、不丢失、不重复。客户端 t1 时刻发送的是对服务器端发送过来的分组的确认，上一条确认信息的序号是 1000，服务器以最大段长 $MSS=1000B$ 发送分组，确认信息当中的确认序号就是所期望收到的下一个数据字节的序号，所以说 t1 时刻这一条确认信息的确认序号应该是 2000；t2 时刻发送数据的报文的序号为 4000，传输 1000B 数据后，序号 4000-4999 都被用掉了，所期望收到的下一个数据字节的序号为 5000。

7. 若客户与服务器建立 TCP 连接的第三次握手时客户端的序号为 1020，当客户向服务器发送了 1000B 的数据后，首先请求与服务器断开连接，则客户收到的第二次握手服务器端的确认序号为（ ）

A. 2019

B. 2020

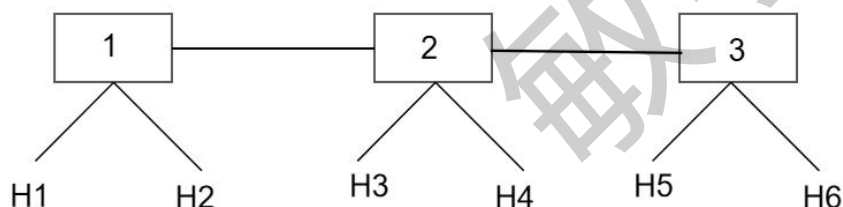
C. 2021

D. 2022

【参考答案】C

【学长点评】考查 TCP 的整个传输流程和序号的变化，要清楚三报文握手和四报文握手的具体细节，序号确认号同步位终止控制位等的变化要求。建立链接时的第三次握手结束后，客户端向服务器端发送数据，因为第三次握手是确认报文段不消耗序号，所以说此时客户端的序号仍然为 1020，发送数据消耗序号，当最后一个传输数据的报文段发送完毕后，序号 2019 已经被用掉了，所以说客户端发送释放报文段时，使用序号 2020，服务器端发送确认报文，确认号是在客户端的序号上加 1，所以说服务器端的确认序号为 2021。

8. 下图所示以太网局域网中，若 H1 与 H2 属于同一个广播域，但不属于同一冲突域，H4 与 H3 不属于同一个广播域，H5 和 H6 属于同一冲突域，则设备 1, 2, 3 分别是（ ）

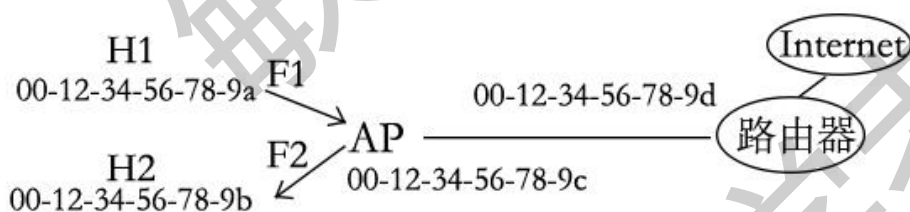


- A. 路由器，交换机，集线器
- B. 交换机，路由器，交换机
- C. 路由器，集线器，交换机
- D. 交换机，路由器，集线器

【参考答案】D

【学长点评】考察物理层、数据链路层和网络层的中继系统的相关知识，主要考察了各个中继系统和冲突域广播域的关系。集线器作为物理层设备，既不能分割冲突域也不能分割广播域，它就像一根接口比较多的网线一样。交换机作为数据链路层设备，只能分割冲突域不能分割广播域。交换机连接的网段仍属于同一个广播域，广播数据包会在交换机连接的所有网段上传播。路由器是网络层设备，既分割了冲突域又分割了广播域。连接到路由器上的网段会被分配成不同的广播域，广播数据不会穿过路由器。设备 3 连接的两个网络都在同一个冲突域，显然设备 3 是集线器；设备 2 连接的两个网络在不同的广播域，所以是路由器；设备 1 连接的两个网络在同一个广播域但不是同一个冲突域，所以是交换机。

9. 在下图所示的网络中，主机 H1 向主机 H2 发送一个 IP 分组，若主机 H2 接收到的 IP 分组的 IEEE802.11 数据帧的 F2，则帧 F2 的地址 1, 2, 3 分别是（ ）



- A. a, b, c
- B. b, a, c
- C. a, c, b
- D. b, c, a

【参考答案】D

【学长点评】考察了 IEEE802.11 数据帧的地址格式要注意分辨两种格式的适用状况。F2 是从接入点 AP 发送到主机 H2，所以说地址 1 是目的地址即主机 H2 地址，地址 2 是 AP 地址，地址 3 是源地址即主机 H1 地址，所以地址 1、2、3 分别是 b, c, a。

去往 AP	来自 AP	地址 1	地址 2	地址 3	地址 4
0	1	目的地址	AP 地址	源地址	-----

1	0	AP 地址	源地址	目的地址	-----
---	---	-------	-----	------	-------

10. 若某无噪声信道带宽为 2MHz，采用 2PSK 调制，则该信道的极限数据传输率为()

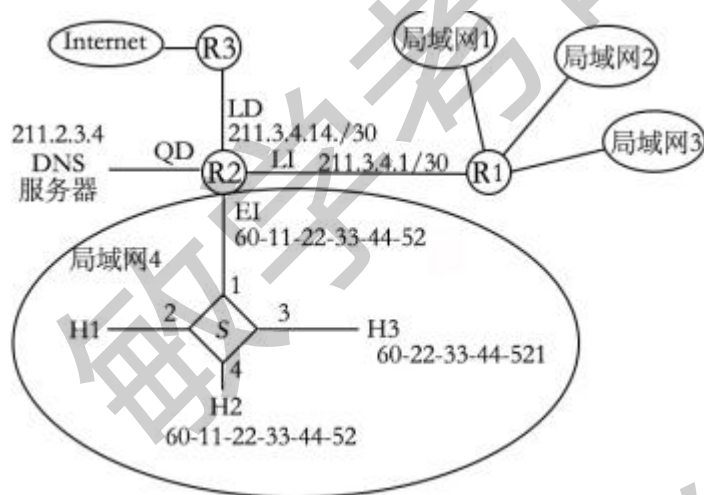
- A. 1Mbps B. 2Mbps C. 4Mbps D. 8Mbps

【参考答案】C

【学长点评】考察基本的带通调制方法和无噪声极限数据传输率的计算。无噪声的情况下，使用奈奎斯特公式进行计算，极大数据传输率 $C=2 \times W \times \log_2(M)$ ，M 是携带数据的码元，可能取的离散值个数(二进制两位离散值个数为 $2^2=4$ ，二进制四位离散值个数为 $2^3=8$ 个)。2PSK(全称：二进制相移键控)是相移键控的最简单的一种形式，它用两个初相相隔为 180 的载波来传递二进制信息。所以也被称为 BPSK。表示信号的两种码元的波形相同，极性相反，也就是说一个码元表示一个二进制数，一共可以表示 2 个离散值， $M=2$ 。 $C=2 \times 2\text{Mhz} \times \log_2(2)=4\text{Mbps}$

二、大题：

某网络拓扑如图，图中 R1，R2，R3 为路由器，其中 R2 运行 DHCP 服务，S 为 100Base-T 的以太网交换机



R2 的路由表结构为

目的网络	子网掩码	下一条	端口
------	------	-----	----

交换机 S 的交换表结构为

MAC 地址	端口
--------	----

(1) 请将 IP 地址空间 201.2.3.0/24, 分配给局域网 124, 要求局域网 1 可以分配不少于 120 个, 局域网 2 和 3 分配不少于 30 个, 局域网局域网 4 不少于 60 个, 请给出分配方案。

(2) 基于 (1) 的分配方案, 给出 R2 路由类, 要求表象尽可能少

(3) 基于 (1) 的分配方案, 为 R2 的 E1 接口分配 IP 地址, 并在下表中为主机 H2 分配至 IP 地址信息

IP 地址	
子网掩码	
默认网关 IP 地址	

(4) 若 S 交换表示空的, H1 接入局域网 4, 并功过 DHCP 请求动态分配 IP 地址, 则 H1 发送的第一个 DHCP 报文是什么报文? 封装该报文的 IP 数据包的元 IP 和目的 IP 分别是什么? 请给出 H1 完成一个 IP 地址配置后的交换机 S 的交换表

【参考答案】

(1) 从 IP 地址的格式可以看出来, 这是无分类域间路由选择 CIDR, 分配地址块时, 主要考虑每个局域网中地址数的个数。根据题目中的要求, 局域网 1 的 IP 地址是 201.2.3.0/25, 局域网 2 和局域网 3 分别是 201.2.3.192/27 和 201.2.3.224/27, 局域网 4 的 IP 地址是 201.2.3.128/26。

(2) 基于 (1) 的分配方案, 给出 R2 路由类, 要求表象尽可能少

目的网络	子网掩码	下一跳	端口
201.2.3.0/25	255.255.255.128	R1	L1
201.2.3.192/27	255.255.255.224	R1	L1
201.2.3.224/27	255.255.255.224	R1	L1
201.2.3.128/26	255.255.255.192	直接交付	E1
其他	默认	R3	L0
211.2.3.4/24 (DNS 服务器 IP 地址, 图片中不完整, 这是假设的)	255.255.255.0	直接交付	E0

(3) 基于 (1) 的分配方案, 为 R2 的 E1 接口分配 IP 地址, 并在下表中为主机 H2 分配至 IP 地址信息

IP 地址	201.2.3.133/26
子网掩码	255.255.255.192
默认网关 IP 地址	即 E1 接口地址

默认网关是离开这个子网所经过的路由器的接口

图片中 IP 地址不太对, 理解 DHCP 分配的是 IP 池中的地址即可, 也就是说分配的是还没有被

(4) 发送的第一个 DHCP 报文是 DHCP 发现报文, 因为此时 H1 还没有被分配 IP 地址, 所以 IP 数据报源 IP 地址为全零即 0.0.0.0, 根据报文规定将目的地址 IP 设为全一即 255.255.255.255

MAC 地址	端口
00-11-22-33-44-56	2
00-11-22-33-44-50	1

MAC 地址 (英语: Media Access Control Address), 直译为媒体存取控制位址, 也称为局域网地址 (LAN Address), MAC 位址, 以太网地址 (Ethernet Address) 或物理地址 (Physical Address). MAC 地址的长度为 48 位 (6 个字节), 通常表示为 12 个 16 进制数, 如: 00-16-EA-AE-3C-40 就是一个 MAC 地址

数据结构

一、选择题:

1. 利用直接插入排序法的思想, 建立一个有序的线性表的时间复杂度为 ()

- A. $O(n)$ B. $O(n \log_2 n)$ C. $O(n^2)$ D. $O(\log_2 n)$

【参考答案】C

【学长点评】利用直接插入的思想, 每插入一个元素的复杂度为 $O(n)$, 建立一个有序线性表的时间复杂度为 $O(n^2)$

2. 将 10 阶对称矩阵压缩存放于一维数组 A 中, 则数组 A 的长度最少为 ()

- A. 100 B. 40 C. 55 D. 80

【参考答案】C

【学长点评】十阶矩阵压缩存放在一维数组中, 且该矩阵为对称矩阵, 则只需要存放上三角元素即可, 共有 55 个元素, 则数组 A 的长度至少为 55.

3. 给定 n 个字符的字符表与字符频率, 霍夫曼编码的时间复杂度为 ()

- A. $O(n)$ B. $O(n \log_2 n)$ C. $O(n^2)$ D. $O(\log_2 n)$

【参考答案】B

【学长点评】对 n 个字符的频率进行排序, 随后构造霍夫曼树, 则时间复杂度便是排序的复杂度, 可为 $O(n \log_2 n)$ 。

4. 用 Prim 算法和 Kruskal 算法构造的最小生成树是 ()

- A. 相同 B. 不同 C. 均可能 D. 无法比较

【参考答案】C

【学长点评】Prim 和 Kruskal 均可以生成最小生成树，且权值相同，但最小生成树可能不同。

5. 散列表有 m 个存储单元，散列函数为 $H(\text{key}) = \text{key} \% p$ ，则 P 的选择（ ）

- A. 小于等于 m 的最大奇数 B. 偶数 C. 素数 D. 合数

【参考答案】C

【学长点评】模的 P 应该为素数。

二、填空题

1. 算术表达式 $a + \{b * (c - d) - e\} / f$ 的后缀表达式为（ ）

【参考答案】 $abcd - * e - f / +$

【学长点评】根据该算术表达式可以得到后缀表达式为 $abcd - * e - f / +$ 。

2. 一棵完全二叉树有 2019 个节点，则该树共有（ ）个叶子结点

【参考答案】1010

【学长点评】共有 2019 个节点，则叶子结点数为 $(2019 + 1) / 2$ 。

3. 包含 n 个顶点和 e 条边的无向图的邻接矩阵中，零元素的个数为（ ）

【参考答案】 $n^2 - 2e$

【学长点评】矩阵共有 n^2 个节点，其中 $2e$ 个节点为非零值，其余元素为 0。

4. 设有向图 G 中有向边的集合 $E = \{ \langle 1, 2 \rangle, \langle 2, 3 \rangle, \langle 1, 4 \rangle, \langle 4, 2 \rangle, \langle 4, 3 \rangle \}$ ，则该图的一个拓扑序列为（ ）

【参考答案】1423

【学长点评】首先 1 号节点无入度，所以第一个遍历，随后删去以 1 号节点开始的边，依次类推，可得拓扑排序为 1423。

三、问答题（10 分）

给你一个后序遍历（比如 6, 7, 5, 10, 11, 9, 8）可以唯一确定一颗二叉树吗？若可以，说明理由。若不可以，则举反例说明。

【参考答案】不可以，后序遍历可知 8 为根节点，但前面元素全属于左子树还是右子树还是全部包含无法知晓，故不可唯一确定一颗二叉树。

四、程序设计题（15 分）

要求：

1. 给出算法的基本设计思想
2. 根据设计思想，给出基本的数据结构

3. 编写算法，栈和队列的基本操作可直接使用。

题目：有两个有序数组 A 和 B，试写出一种尽可能高效的算法找出序列中第 k 小的元素。并说明你所设计的算法的时间复杂度和空间复杂度。

【学长点评】有 $O(k)$ 的复杂度算法，此为基础解，还有更低的算法设计，复杂度为 $O(\log m + \log n)$

【参考答案】基础解：

```
int findKthSMallest(int[] A, int[] B, int k) {
    int a_offset = 0, b_offset = 0;
    if (A.length + B.length < k) return -1;

    while (true) {
        if (a_offset < A.length) {
            while (b_offset == B.length ||
                A[a_offset] <= B[b_offset]) {
                a_offset++;
                if (a_offset + b_offset == k) return A[a_offset];
            }
        }
        if (b_offset < B.length) {
            while (a_offset == A.length ||
                A[a_offset] >= B[b_offset]) {
                b_offset++;
                if (a_offset + b_offset == k) return B[b_offset];
            }
        }
    }
}
```

标准解：

```
int findKthSmallest(int A[], int m, int B[], int n, int k) {
    assert(m >= 0); assert(n >= 0); assert(k > 0); assert(k <= m+n);

    int i = (int)((double)m / (m+n) * (k-1));
    int j = (k-1) - i;

    assert(i >= 0); assert(j >= 0); assert(i <= m); assert(j <= n);
```



```
// invariant: i + j = k-1
// Note: A[-1] = -INF and A[m] = +INF to maintain invariant
int Ai_1 = ((i == 0) ? INT_MIN : A[i-1]);
int Bj_1 = ((j == 0) ? INT_MIN : B[j-1]);
int Ai    = ((i == m) ? INT_MAX : A[i]);
int Bj    = ((j == n) ? INT_MAX : B[j]);

if (Bj_1 < Ai && Ai < Bj)
    return Ai;
else if (Ai_1 < Bj && Bj < Ai)
    return Bj;

assert((Ai > Bj && Ai_1 > Bj) ||
       (Ai < Bj && Ai < Bj_1));

// if none of the cases above, then it is either:
if (Ai < Bj)
    // exclude Ai and below portion
    // exclude Bj and above portion
    return findKthSmallest(A+i+1, m-i-1, B, j, k-i-1);
else /* Bj < Ai */
    // exclude Ai and above portion
    // exclude Bj and below portion
    return findKthSmallest(A, i, B+j+1, n-j-1, k-j-1);
```